Práctica 2. Calculadora simple

Manuela Planelles Lucas

1º DAW – 2024/25 – Programación

1. Objetivo de la practica 2
2. Programa en Java 2
2.1. Pasos seguidos 2
2.2. Explicación del código 2
3. Plan de pruebas 4
3.1. Prueba con texto y números negativos 4
3.2. Prueba introduciendo ceros 4
3.3. Prueba con cero en el denominador en el cado de la división 5
3.4. Prueba cambiando el formato de entrada del signo 5
3.5. Prueba correcta – caso feliz 6

GitHub

1. Objetivo de la practica

El objetivo es implementar en Java una calculadora simple que permita al usuario realizar operaciones de suma, resta, multiplicación, división y raíz cuadrada. El programa debe usar switch-case para gestionar las operaciones, un bucle do-while para repetir mientras el usuario no indique que quiere salir, y controlar las excepciones que se produzcan cuando se introduzcan datos incorrectos.

2. Programa en Java

2.1. Pasos seguidos

Lo primero fue estructurar el flujo del programa: pedir el primer operando, mostrar el menú de operaciones y pedir la operación controlando mayúsculas y minúsculas con toUpperCase(). Decidí pedir el segundo operando dentro de cada case que lo necesita, en lugar de hacerlo fuera con un if, porque así cada operación controla exactamente lo que necesita y el default puede capturar errores antes de pedir datos innecesarios.

Usé switch-case para gestionar cada operador de forma independiente, de manera que cada case controla su propia acción sin interferir con el resto.

Añadí un bucle do-while para que el programa se repita hasta que el usuario escriba "SALIR", permitiendo hacer varias operaciones seguidas sin tener que relanzar el programa.

Por último, envolví todo en un try-catch para controlar la excepción que lanza Java cuando el usuario introduce una letra donde se espera un número.

2.2. Explicación del código

operacion se declara fuera del do-while como "" para que el while pueda comparar desde la primera iteración sin que salte una excepción.

El do-while contiene todo el flujo. Se repite mientras la operación no sea "SALIR".

El try-catch envuelve todo el bloque de lectura y cálculo. Captura InputMismatchException, que es el error que lanza Java cuando se introduce una letra donde se espera un número. El teclado.next() dentro del catch limpia el buffer para que el bucle no se quede atascado.

El segundo operando se pide dentro de cada case que lo necesita. De esta forma el default captura cualquier operación no válida antes de pedir datos innecesarios, y la raíz cuadrada y el "SALIR" simplemente no tienen esa línea en su case.

El switch evalúa la operación con toUpperCase() para que no distinga entre mayúsculas y minúsculas. Cada case calcula y muestra el resultado. El default muestra error si el signo no es válido.

En la división se controla que el segundo operando no sea cero antes de operar.

```java
public class calculadoraSimple {  MANUELA\manue *
    public static void main(String[] args) {  MANUELA\manue *
        Scanner teclado = new Scanner(System.in);
        String operacion = "";

        System.out.println("*** BIENVENIDO A LA CALCULADORA RÁPIDA ***");

        do{
            try{
                System.out.println("Introduce el operador: ");
            int operador1 = teclado.nextInt();
            System.out.println("----------------------------");
            System.out.println("[+] -> sumar");
            System.out.println("[-] -> resta");
            System.out.println("[X] -> multiplicar");
            System.out.println("[/] -> dividir");
            System.out.println("[R] -> raíz cuadrada");
            System.out.println("[Salir]");
            System.out.println("----------------------------");
            System.out.println("Introduce el operador: ");
            operacion = teclado.next().toUpperCase();

            switch (operacion){
                case "+":
                    System.out.println("Introduce el segundo operando: ");
                    int operador2 = teclado.nextInt();
                    System.out.println("El resultado de " + operador1 + " + " + operador2 + " es " + (operador1 + operador2));
                    break;
                case "-":
                    System.out.println("Introduce el segundo operando: ");
                    int operador3 = teclado.nextInt();
                    System.out.println("El resultado de " + operador1 + " - " + operador3 + " es " + (operador1 - operador3));
                    break;
                case "X":
                    System.out.println("Introduce el segundo operando: ");
                    int operador4 = teclado.nextInt();
                    System.out.println("El resultado de " + operador1 + " x " + operador4 + " es " + (operador1 * operador4));
                    break;
                case "/":
                    System.out.println("Introduce el segundo operando: ");
                    int operador5 = teclado.nextInt();
                    if (operador5 == 0) {
                        System.out.println("ERROR, no se puede dividir entre 0");
                    } else {
                        double divi = (double) operador1 / operador5;
                        System.out.println("El resultado de " + operador1 + " / " + operador5 + " es " + divi);
                    }
                    break;
                case "R":
                    double resul = Math.sqrt(operador1);
                    System.out.println("La raíz cuadrada de " + operador1 + " es " + resul);
                    break;
                case "SALIR":
                    System.out.println("¡Hasta luego!");
                    break;
                default:
                    System.out.println("ERROR, la operación no es valido");
                    break;
            }
            } catch (InputMismatchException e) {
                System.out.println("Debes introducir un número.");
                teclado.next();
            }
        }while (!operacion.equals("SALIR"));

    }
}
```

3. Plan de pruebas

3.1. Prueba con texto y números negativos

Cuando se introduce una letra como primer operando, el catch captura el error, muestra el aviso y se volver a pedir el número correctamente.

Cuando se introduce un número negativo, el programa lo acepta y opera con él sin ningún problema.

```
*** BIENVENIDO A LA CALCULADORA RÁPIDA ***
Introduce el operador:
a
Debes introducir un número.
Introduce el operador:
-5
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
```

3.2. Prueba introduciendo ceros

Cuando se introduce 0 como operando, el programa lo acepta y calcula el resultado correctamente. El único caso donde el cero da problemas es en la división.

```
*** BIENVENIDO A LA CALCULADORA RÁPIDA ***
Introduce el operador:
0
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
+
Introduce el segundo operando:
0
El resultado de 0 + 0 es 0
Introduce el operador:
```

3.3. Prueba con cero en el denominador en el cado de la división

Cuando se elige la división e introduce 0 como segundo operando, el programa detecta que no se puede dividir entre cero y muestra el mensaje de error sin llegar a calcular nada.

```
*** BIENVENIDO A LA CALCULADORA RÁPIDA ***
Introduce el operador:
8
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
/
Introduce el segundo operando:
0
ERROR, no se puede dividir entre 0
Introduce el operador:
```

3.4. Prueba cambiando el formato de entrada del signo

Cuando se introduce el signo en minúscula, por ejemplo x en lugar de X, el toUpperCase() lo convierte automáticamente y el programa funciona con normalidad.

```
Introduce el operador:
5
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
x
Introduce el segundo operando:
2
El resultado de 5 x 2 es 10
```

Cuando se introduce un signo que no existe, por ejemplo *, el programa entra en el default del switch y muestra el mensaje de error.

```
*** BIENVENIDO A LA CALCULADORA RÁPIDA ***
Introduce el operador:
2
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
*
ERROR, la operación no es valido
Introduce el operador:
```

3.5. Prueba correcta – caso feliz

Cuando se introduce dos números válidos y elige cada una de las operaciones disponibles, el programa calcula y muestra el resultado correctamente en todos los casos. Al escribir "SALIR" el programa muestra el mensaje de despedida y termina.

```
*** BIENVENIDO A LA CALCULADORA RÁPIDA ***
Introduce el operador:
5
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
+
Introduce el segundo operando:
4
El resultado de 5 + 4 es 9
```

```
Introduce el operador:
5
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
-
Introduce el segundo operando:
4
El resultado de 5 - 4 es 1
```

```
Introduce el operador:
7
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
x
Introduce el segundo operando:
2
El resultado de 7 x 2 es 14
```

```
Introduce el operador:
8
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
/
Introduce el segundo operando:
2
El resultado de 8 / 2 es 4.0
```

```
Introduce el operador:
8
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
r
La raíz cuadrada de 8 es 2.8284271247461903
```

```
Introduce el operador:
8
----------------------------
[+] -> sumar
[-] -> resta
[X] -> multiplicar
[/] -> dividir
[R] -> raíz cuadrada
[Salir]
----------------------------
Introduce el operador:
salir
¡Hasta luego!

Process finished with exit code 0
```